Core/Src/main.c

```
/* CPE 3160
 * John Oliver
 *
 * A5 - SPI Digital Analog Converter
 * Chris Amey and Alan Kusparmakov
 *
 * Analog Discovery Reading:
 * https://drive.google.com/file/d/1ridWuwsIxUjhIY48B9w1fNhTV8Lpmowz/view?usp=sharing

 * SPI data transmission example:
 * https://drive.google.com/file/d/1ScAAHVJ-xHwXpT-e0M0DIUzHLanU-KGV/view?usp=sharing
 *
 * The header 0x3 contains dac information.
 * Then there is this message 0b111000111000. No meaning.
 *
 *
 * We then sent over the codes corresponding to 1 and 2V
 * 25% duty cycle between these values:
 * https://drive.google.com/file/d/1OmJZAfnXotZHQ4mua-e7QsWWCk10DQcZ/view?usp=sharing


 And of course the video!!!!!!!1
 https://drive.google.com/file/d/1OmJZAfnXotZHQ4mua-e7QsWWCk10DQcZ/view?usp=sharing

 */

#include "stdbool.h"
#include "stm32l476xx.h"
#include "stm32l4xx_hal.h"
#include "stm32l4xx_hal_gpio.h"
#include "stm32l4xx_hal_gpio_ex.h"
#include <stddef.h>
#include <stdint.h>

typedef enum { ALTERNATIVE = 2 } GpioMode;

typedef enum GpioState { HIGH = 1, LOW = 0 } GpioState;

// configure microcontroller clock output on PA8
void set_PA8_clock_MSI()
{
  // unselect all clocks
  // I think "selection" is just for blanket-setting an option to multiple clocks if you need to
  RCC->CFGR = ((RCC->CFGR & ~(RCC_CFGR_MCOSEL)));

  // select 0010 = MSI system clock
  RCC->CFGR |= RCC_CFGR_MCOSEL_0;

  // now configure selected clock(s)

  // enable GPIOA
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

  // set mode
  GPIOA->MODER &= ~GPIO_MODER_MODE8_Msk;
  GPIOA->MODER |= (ALTERNATIVE << GPIO_MODER_MODE8_Pos);

  // allows STM to actively drive either low or high
  GPIOA->OTYPER &= ~(GPIO_OTYPER_OT8); // Push-pull output

  //
  GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD8); // no resistor
  GPIOA->OSPEEDR |= (GPIO_OSPEEDR_OSPEED8); // high speed

  // Set Pin 8's Alternate Function to be MCO
  GPIOA->AFR[1] &= ~(GPIO_AFRH_AFSEL8); // select MCO function
}

void TIM2_init()
{
  // APB1 = advanced peripheral bus
  // enable the CPU to TIM2 connection for configuring.
  // also connects a peripheral clock to TIM2
  RCC->APB1ENR1 |= (RCC_APB1ENR1_TIM2EN_Msk);

  // disable timer while configuring
  TIM2->CR1 &= ~(TIM_CR1_CEN_Msk);

  // CR1.DIR is up count by default
  // this is what we want

  TIM2->PSC = 0;
  TIM2->ARR = 799;

  // start at a known val
  TIM2->CNT = 0;

  // enable update interrupt
  // goes off whenever CNT == ARR
  TIM2->DIER |= TIM_DIER_UIE_Msk;

  // enable CCR1
```

```
  // goes off whenever CNT = CCR1
  TIM2->DIER |= TIM_DIER_CC1IE;

  // set the value to 25% of the duty cycle (800 / 4)
  TIM2->CCR1 = 200;

  // initialize the update interrupt to zero
  TIM2->SR &= ~(TIM_SR_UIF_Msk);

  // enable TIM2 in the NVIC
  // TIM2 is ISER[0] within NVIC.
  NVIC->ISER[0] = 1 << TIM2_IRQn;

  // can also do this
  // NVIC_EnableIRQ(TIM2_IRQn);

  // re-enable timer
  TIM2->CR1 |= TIM_CR1_CEN_Msk;

  // terminology:
  // DIER enable timer interrupt requests?
  // EGR generate timer events by software?
  // CR counter enable
  // UIE - update interrupt enable
  // CEN - counter enable
}

void set_PA0_wave_output()
{
  // enable GPIOA
  // here I am redundantly setting this flag so that there isn't a central
  // gpio init function that other functions rely on. either works for
  // proof of concept
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

  // output mode
  GPIOA->MODER &= ~GPIO_MODER_MODE0_Msk;
  GPIOA->MODER |= 0b01 << GPIO_MODER_MODE0_Pos;

  // no push or pull resistors
  GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0_Msk;
  GPIOA->PUPDR |= 0b00 << GPIO_PUPDR_PUPD0_Pos;

  // output push-pull so that the pin drives 0 actively and 1 actively
  GPIOA->OTYPER &= ~GPIO_OTYPER_OT0_Msk;
  GPIOA->OTYPER |= 0b0 << GPIO_OTYPER_OT0_Pos;
}

void set_PA0(GpioState state)
{
  // GPIOA->ODR &= ~GPIO_ODR_OD0_Msk;
  // GPIOA->ODR |= state << GPIO_ODR_OD0_Pos;

  // the above can apparently be achieved also with a bit set-reset register
  // notice that we're setting the BSRR directly rather than with &= or |=
  // this is becasue BSRR is a write only register
  if (state == HIGH)
    GPIOA->BSRR = GPIO_BSRR_BS0;
  else
    GPIOA->BSRR = GPIO_BSRR_BR0;
}

void SPI_init()
{
  // GPIO clocks
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

  // SPI clock
  RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

  // PA5 and PA7 alternative mode
  // STM32 defaults:
  // PA5 = SCK
  // PA6 = MISO = POCI
  // PA7 = MOSI = PICO
  GPIOA->MODER &= ~(GPIO_MODER_MODE5_Msk | GPIO_MODER_MODE7_Msk);
  GPIOA->MODER |= 0b10 << GPIO_MODER_MODE5_Pos;
  GPIOA->MODER |= 0b10 << GPIO_MODER_MODE7_Pos;

  // AF=5=SPI for PA5 and PA7
  GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL5_Msk | GPIO_AFRL_AFSEL7_Msk);
  GPIOA->AFR[0] |= GPIO_AF5_SPI1 << GPIO_AFRL_AFSEL5_Pos;
  GPIOA->AFR[0] |= GPIO_AF5_SPI1 << GPIO_AFRL_AFSEL7_Pos;

  GPIOB->MODER &= ~(GPIO_MODER_MODE6_Msk);
  GPIOB->MODER |= (0b01 << GPIO_MODER_MODE6_Pos);

  // Enable GPIOB clock (if not already enabled)
  RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

  // PB6: General-purpose output (01)
  GPIOB->MODER &= ~(GPIO_MODER_MODE6_Msk);
  GPIOB->MODER |= (0b01 << GPIO_MODER_MODE6_Pos);

  // PB6: Push-pull output (0)
```

```
  GPIOB->OTYPER &= ~(1U << 6);

  // PB6: No pull-up or pull-down (00)
  GPIOB->PUPDR &= ~(GPIO_PUPDR_PUPD6_Msk);

  // PB6: Low-speed output (00)
  GPIOB->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED6_Msk); // ?

  // Initialize chip select as high (idle)
  GPIOB->BSRR = GPIO_BSRR_BS6;

  // disable SPI while configuring
  SPI1->CR1 &= ~SPI_CR1_SPE_Msk;

  // ----
  // CR1 config
  SPI1->CR1 |= SPI_CR1_MSTR;

  // baud rate calculation
  SPI1->CR1 &= ~(SPI_CR1_BR_Msk);
  SPI1->CR1 |= 1 << SPI_CR1_BR_Pos;

  // CPOL = 0 = idle low
  SPI1->CR1 &= ~(SPI_CR1_CPOL);

  // CPHA = 0 = rising edge sample
  SPI1->CR1 &= ~(SPI_CR1_CPHA);

  // LSBFIRST = 0 => Most Significant Bit first
  SPI1->CR1 &= ~(SPI_CR1_LSBFIRST);

  // SSM = 1 = NSS is managed by software through SSI instead of hardware
  SPI1->CR1 |= SPI_CR1_SSM;

  // SSI = 1 = internal NSS kept high in controller mode
  SPI1->CR1 |= SPI_CR1_SSI;

  // 16 bit frame
  SPI1->CR2 &= ~SPI_CR2_DS_Msk;
  SPI1->CR2 |= (0b1111 << SPI_CR2_DS_Pos);

  // -----------
  // re-enable SPI
  SPI1->CR1 |= SPI_CR1_SPE;
}

void SPI_send_data(uint16_t value)
{
  // header may be different depending on the target

  // A/B = 0 = DAC A
  // BUF = 0 = unbuffered VREF
  // GA = 1 = 1x gain
  // SHDN = 1 enable output

  // TXE = 0: transmit buffer is not empty
  // TXE = 1: transmit buffer is empty

  // == 0b0011

  uint16_t command = 0x3 << 12 | (value & 0x0FFF);

  // CS low = begin transmission
  // PB6 is CS in STM32
  // BSRR modifies ODR
  GPIOB->BSRR = GPIO_BSRR_BR6;

  // wait until transmit buffer is empty
  while (!(SPI1->SR & SPI_SR_TXE))
    ;

  SPI1->DR = command;

  // wait until the transmit buffer is empty.
  // if we don't do this, we might check the BSY flag before
  // any data actually entered SPI, which would be result in a
  // premature end of transmission
  while (!(SPI1->SR & SPI_SR_TXE))
    ;

  // after the data has exited the buffer, check that SPI is done
  // processing it.
  while (SPI1->SR & SPI_SR_BSY)
    ;

  // B6 high = CS high = end transmission
  GPIOB->BSRR = GPIO_BSRR_BS6;
}

uint16_t SPI_volt_convert(float voltOut) {
    //Calculate the output voltage
    float gain = 1.0;
    float DAC_value = (voltOut * 4096.0f) / (3.3f * gain);

    //Clamp to valid range
    if (DAC_value < 0.0) {
```

```
        DAC_value = 0.0;
    }

    if (DAC_value > 4095.0) {
        DAC_value = 4095.0;
    }

    //Cast the data to valid type and return
    uint16_t ans = (uint16_t)(DAC_value);
    return ans;
}

int main()
{
  HAL_Init();

  SPI_init();

  TIM2_init();


  while (true) {
    // SPI_send_data(0b111111111111);
    // HAL_Delay(1000);
    // SPI_send_data(0b000000000000);
    // HAL_Delay(1000);

    // SPI_send_data(0b)
  }
}
//
void TIM2_IRQHandler()
{
  if (TIM2->SR & TIM_SR_UIF) {
    uint16_t SPI_code = SPI_volt_convert(2);
    SPI_send_data(SPI_code);

    // SPI_send_data(0xFFF);

    // always clear flag
    TIM2->SR &= ~(TIM_SR_UIF);
    return;
  }

  if (TIM2->SR & TIM_SR_CC1IF) {
    uint16_t SPI_code = SPI_volt_convert(1);
    SPI_send_data(SPI_code);
    // SPI_send_data(0x0);

    // clear flag
    TIM2->SR &= ~(TIM_SR_CC1IF);
    return;
  }
}
```

Core/CMakeLists.txt

```
target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE Inc)
add_subdirectory(Src)
```

Core/Inc/main.h

```c
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.h
  * @brief          : Header for main.c file.
  *                   This file contains the common defines of the application.
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

/* Includes ------------------------------------------------------------------*/
#include "stm32l4xx_hal.h"
#include "stm32l4xx_nucleo.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Exported types ------------------------------------------------------------*/
/* USER CODE BEGIN ET */

/* USER CODE END ET */

/* Exported constants --------------------------------------------------------*/
/* USER CODE BEGIN EC */

/* USER CODE END EC */

/* Exported macro ------------------------------------------------------------*/
/* USER CODE BEGIN EM */

/* USER CODE END EM */

/* Exported functions prototypes ---------------------------------------------*/
void Error_Handler(void);

/* USER CODE BEGIN EFP */

/* USER CODE END EFP */

/* Private defines -----------------------------------------------------------*/
#define USART_TX_Pin GPIO_PIN_2
#define USART_TX_GPIO_Port GPIOA
#define USART_RX_Pin GPIO_PIN_3
#define USART_RX_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB

/* USER CODE BEGIN Private defines */

/* USER CODE END Private defines */

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

Core/Src/CMakeLists.txt

```
target_sources(${CMAKE_PROJECT_NAME} PRIVATE main.c)
```